

On the non-uniform semantics of *return-type-requirements*

Document number: P1452R0
Date: 2019-01-21
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup: Evolution
Revises: None
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Introduction

What we have as a *return-type-requirement* today is a misnomer in both the syntactic and semantic sense. The Concepts TS introduced two kinds of semantic constraints that syntactically took the form of a *trailing-return-type*: the deduction constraint and the implicit conversion constraint. In the time since, deduction constraints were first changed by the lack of constrained placeholders until the adoption of P1141R1 at the 2018 San Diego meeting (becoming syntactically different from a *trailing-return-type*), and then—with P1084R2—they ceased to deduce in the manner associated with return type deduction. The syntactic space occupied by deduction constraints in the Concepts TS is now occupied by something else altogether; however, the implicit conversion constraint lives on. Yet the two are both *return-type-requirements*, using the `->` token.

The inconsistency between the uses of the `->` token in the same context seems unfortunate; and the *trailing-return-type* form of *return-type-requirement*, with its current semantics, appears to be easily replaceable. Its presence in the language seems to be an unnecessary complication that may frustrate future extensions.

`-> Type` versus `-> Concept`

The `-> Type` form of *return-type-requirement* behaves consistently with the behaviour of `return E`; for a function that is declared to return *Type* except that placeholder types are allowed for *trailing-return-types* in function declarations and not for *return-type-requirements*. That is, the semantics of the various *compound-requirements* in

```
requires {  
    { E } -> Type;  
    { [](Type) {}(E) };  
    { E } -> ConvertibleTo<Type>;  
};
```

are roughly the same. The last form is less aware of the context in terms of access checking, null pointer conversion, and other cases where perfect forwarding is less-than-perfect. This last form is also the form used by the library in N4791, the post-San Diego working draft. Note, however, that `Same` is more often used than `ConvertibleTo` in N4791.

In contrast, the `-> Concept` form of *return-type-requirement* most notably does not use a type after the `->` token. It also deduces in a manner different from either of `Concept auto` or `Concept decltype(auto)` would for a function whose declared return type contains such a placeholder type, and it does not involve a check for convertibility.

Extending the current `-> Type` case to allow placeholder types would leave us with a problem where the extension of the current semantics would lead to a difference in the meaning of the first two *compound-requirements* in

```
requires {  
    { E } -> Concept;  
    { E } -> Concept auto;  
    { [](Concept auto) {}(E) };  
};
```

However, we note that the third *compound-requirement* already expresses a deduction constraint (except that the TS wording causes access checking, etc. to be ignored). It would seem that extending from the current semantics of `-> Type` is not very profitable.

Possibility for a more powerful `-> Type`

Today's placeholder types are limited in context; however, the Concepts TS allowed deduction constraints like

```
-> std::vector<Boolean>
```

We note that this sort of “type pattern” is expressed in the language as a type. Thus, if we retain `-> Type` with its current semantics, we will not gain the benefit of applying P1084 to such cases if we were to adopt them into the language.

We also note that, if we performed the template argument deduction upon `decltype(E)` (while requiring that the deduced *A* is identical to *A*) in the style of partial specialization matching instead of applying template argument deduction upon the expression in the style of a call to a function template, then `-> Type` would have semantics matching that of `-> Same<Type>`. Additional benefits are that `-> Concept auto` would work like `-> Concept` does in N4791, and that replacing a concrete type in a *return-type-requirement* with a concept that the type models would not cause the *compound-requirement* to reject previously accepted cases.

Proposal for C++20: remove `-> Type`

The `-> Type` form of *return-type-requirement* is underpowered and does not bring additional expressiveness to the language. It is on the wrong side of the split between `ConvertibleTo` and `Same` as used with *compound-requirements* in N4791, and it introduces complications for future extensions; therefore, it is proposed that the *trailing-return-type* form of *return-type-requirement* be removed. At the same time, it is proposed that *trailing-type-requirement* is a more appropriate name.